

Rapport hebdomadaire : Fine-tuning pour segmentation prosodique

TROMBATI Mohamed

Semaine du 15 au 19 juin 2026

1 Objectif

Cette semaine a été consacrée à la mise en place d’un pipeline de fine-tuning pour la segmentation prosodique. L’objectif est d’entraîner une tête de segmentation au-dessus d’un encodeur de parole gelé sur le corpus Common Voice FR.

Le but est de comparer les performances obtenues avec les résultats de l’approche zero-shot de la semaine précédente.

2 Méthodologie

2.1 Pipeline général

Le pipeline d’apprentissage est structuré en trois étapes :

- extraction des embeddings à partir d’un encodeur de parole gelé,
- génération automatique de labels faibles à partir de signaux acoustiques,
- entraînement d’une tête de segmentation supervisée.

2.2 Extraction des embeddings

Chaque audio du corpus Common Voice est traité de la manière suivante :

- chargement à 16 kHz,
- passage dans l’encodeur gelé (pour cette semaine j’ai pu seulement travailler sur `Pantagruel-B-1K`),
- extraction des représentations temporelles (last hidden states),
- stockage des embeddings dans un cache disque.

Cette étape permet d’éviter de recalculer les représentations à chaque epoch.

2.3 Génération des labels

Les labels de segmentation sont construits automatiquement à partir de pauses détectées dans le signal.

Une frontière est définie lorsqu’une pause dépasse un seuil de durée de 0,25 s (ce seuil ainsi que d’autres seuils sont modifiables dans le fichier `configs/config_finetuning.yaml`).

2.3.1 Labels continus

Afin de modéliser l’incertitude temporelle des frontières, une représentation lissée est utilisée :

$$\ell[t] = \max_b \exp \left(-\frac{(t - t_b)^2}{2\sigma^2} \right) \quad (1)$$

Cette formulation permet de représenter une frontière comme une zone de probabilité plutôt qu'un point strict.

Les valeurs sont ensuite normalisées dans $[0, 1]$.

Remarque : la normalisation probabiliste a été retirée afin d'éviter des valeurs supérieures à 1.

3 Architecture du modèle

Le modèle entraîné est composé de :

- normalisation LayerNorm des embeddings,
- deux couches convolutionnelles 1D,
- un BiLSTM bidirectionnel,
- une couche linéaire de classification par frame.

L'encodeur reste entièrement gelé.

4 Entraînement

4.1 Configuration

Paramètre	Valeur
Optimiseur	AdamW
Learning rate	3e-4
Loss	BCEWithLogitsLoss
Batch size	8
Gradient clipping	1.0
Early stopping	patience = 5

4.2 Gestion des séquences

Les séquences étant de longueurs variables, un padding dynamique est appliqué. Les positions paddées sont ignorées lors du calcul de la loss.

5 Résultats du fine-tuning

L'entraînement de la tête de segmentation a été effectué sur 14 epochs avec early stopping. Les performances sur le jeu de validation sont résumées ci-dessous.

5.1 Analyse des résultats

L'entraînement montre une convergence rapide de la loss d'entraînement, passant de 0.1345 à 0.0158, indiquant que la tête de segmentation apprend efficacement à exploiter les représentations de l'encodeur gelé.

La loss de validation atteint un minimum à **0.0197 (epoch 9)**, qui correspond également au meilleur modèle sauvegardé.

Le score F1 de validation atteint un maximum de **0.3044**, obtenu à l'epoch 10, ce qui suggère une stabilité relative des performances autour de 0.29–0.30 après convergence.

Epoch	Train Loss	Valid Loss	Valid F1
1	0.1345	0.0418	0.2335
2	0.0318	0.0251	0.2311
3	0.0245	0.0233	0.2905
4	0.0222	0.0212	0.2703
5	0.0210	0.0215	0.2528
6	0.0203	0.0208	0.2893
7	0.0197	0.0206	0.3018
8	0.0191	0.0201	0.2959
9	0.0184	0.0197	0.2954
10	0.0177	0.0208	0.3044
11	0.0172	0.0206	0.2954
12	0.0168	0.0200	0.3008
13	0.0164	0.0214	0.2906
14	0.0158	0.0233	0.2836

TABLE 1 – Suivi des performances du modèle **Pantagruel-B-1K** de segmentation pendant le fine-tuning (early stopping à l’epoch 14).

On observe également un léger phénomène de surapprentissage après l’epoch 10, avec une augmentation progressive de la validation loss malgré une diminution continue de la training loss.

5.2 Conclusion intermédiaire

Ces résultats confirment que le fine-tuning améliore légèrement les performances par rapport à une approche zero-shot, tout en restant limité par la qualité des pseudo-labels utilisés pour la supervision.

6 Bugs rencontrés

6.1 Erreur d’ordre des arguments

Une inversion entre `duration` et `num_frames` a provoqué une erreur dans la génération des labels. Le problème a été corrigé en utilisant des arguments nommés.

6.2 Labels hors intervalle

Une erreur dans la formulation de la gaussienne introduisait un facteur de normalisation incorrect, produisant des valeurs supérieures à 1.

La correction consiste à utiliser :

$$\exp\left(-\frac{(t - t_b)^2}{2\sigma^2}\right) \quad (2)$$

sans normalisation.

7 État actuel

Le pipeline est désormais fonctionnel et l’entraînement est en cours sur **Pantagruel-B-1K**.

- extraction des embeddings terminée,
- labels générés automatiquement,
- entraînement en cours,
- évaluation non encore effectuée.

8 Perspectives

Les prochaines étapes sont :

- évaluation du modèle fine-tuné,
- comparaison avec le benchmark zero-shot,
- extension aux autres modèles (**Pantagruel**, **LeBenchmark**),
- analyse des gains du fine-tuning.

9 Conclusion

Ce travail met en place un pipeline complet de fine-tuning pour la segmentation prosodique. Il constitue une extension directe du benchmark zero-shot précédent et permettra de quantifier l'impact de l'adaptation supervisée sur les représentations auto-supervisées.